Множественное наследование

Множественное наследование

```
class A {
public:
    int x = 2;
};


class B {
public:
    int y = 4;
};


class C: public A,  B {
public:
    int z = 5;
};
```

Расположение в памяти:

x	y	z

```
int main() {
    C c;
    c.x = 3;  //ок
    c.y = 6;  //Ошибка, private наследование
}
```

Пример

```
class A {
public:
    int x = 2;
};
class B {
public:
    int y = 4;
};
class C: public A,  B {
public:
    int z = 5;
};
void f (A a) {}
void f1 (B b) {}
int main() {
    C c;
    c.x = 3;
    f(c);           //Ok
    f1(c);          //Private наследование, запрещено
    A* pa = &c;
    std::cout << pa->x << std::endl;  //3
}
```

Одноимённые методы в базовых классах

```
class A {
public:
    int x = 2;
    void f() {std::cout << "A\n";}
};
class B {
public:
    int y = 4;
    void f() {std::cout << "B\n";}
};
class C: public A,  public B {
public:
    int z = 5;
};
void f (A a) {}
void f1 (B b) {}
int main() {
    C c;
    c.A::f();
    c.B::f();
}
```

Указатель на метод

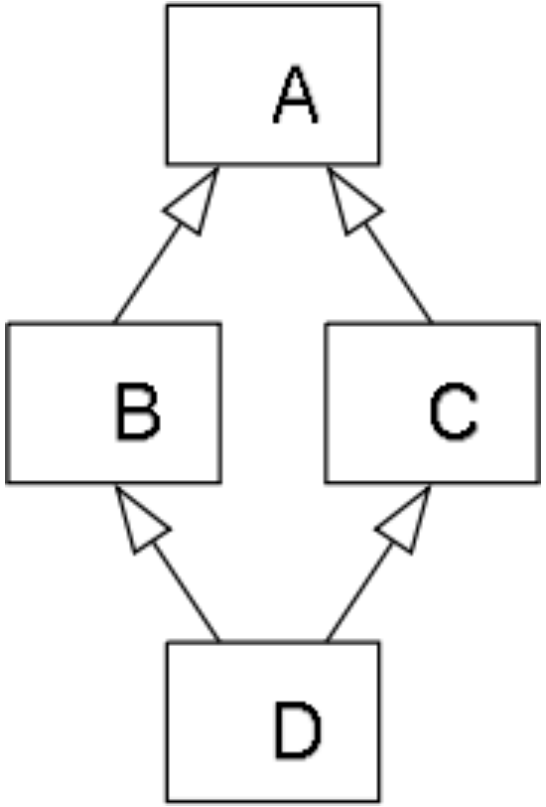
```
class A {
public:
    int x = 2;
    void f(){std::cout << "A\n";}
};
class B {
public:
    int y = 4;
    void f() {std::cout << "B\n";}
};
class C: public A,  public B {
public:
    int z = 5;
    void f() { std::cout << "C\n";}
};

int main() {
    void (A::*p)() = &A::f;
    C c;
    (c.*p)();
}
```

Проблема ромба

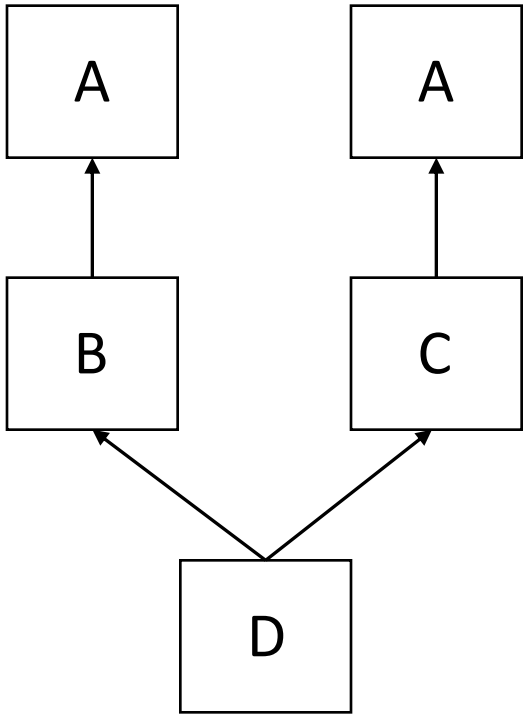
```
class A {
public:
    int a = 2;
    void f(){std::cout << ”A\n";}
};
class B: public A {
public:
    int b = 2;
};
class C: public A {
public:
    int c = 4;
};
class D: public B,  public C {
public:
    int d = 5;
};


int main() {
    D d;
    d.f();     //неоднозначность
    d.A::f(); //неоднозначность
}
```



Ромбовидное наследование (продолжение)

```
class A {
public:
    int a = 2;
    void f(){std::cout << "A\n";}
};
class B: public A {
public:
    int b = 2;
};
class C: public A {
public:
    int c = 4;
};
class D: public B,  public C {
public:
    int d = 5;
};
int main() {
    D d;
    d.C::f();  // ok
    A& a = d;  // Ambiguous conversion
}
```



Расположение в памяти:

Пример

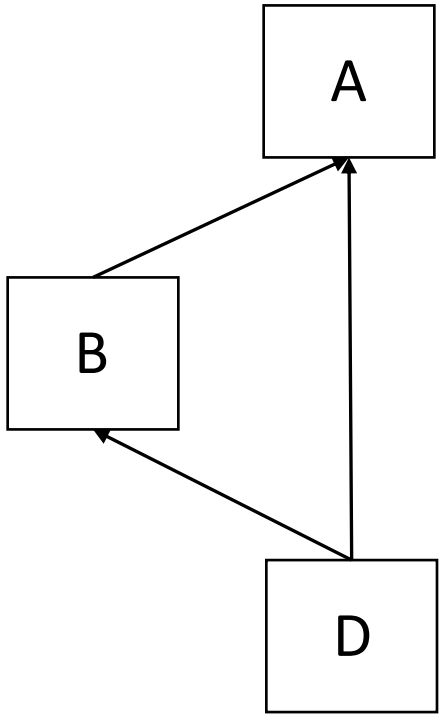
```
class A {
public:
    void f(){std::cout << "A\n";}
};
class B: public A {
public:
    void f(){std::cout << "B\n";}
};
class C: public A {
public:
    void f(){std::cout << "C\n";}
};
class D: public B,  public C {
public:
    int d = 5;
    void f(){std::cout << "D\n";}
};

int main() {
    D d;
    d.f();       //D
    d.B::f();    //B
    d.C::f();    //C
    static_cast<B>(d).A::f();   //A
    static_cast<C>(d).A::f();   //A
}
```

Direct base 'A' is inaccessible due to ambiguity

```
class A {
public:
    void f(){std::cout << "A\n";}
};
class B: public A {
public:
    void f(){std::cout << "B\n";}
};
class D: public B,  public A {
public:
    int d = 5;
    void f(){std::cout << "D\n";}
};


int main() {
    D d;
    d.f();       //D
    d.B::f();    //B
    static_cast<B>(d).A::f();   //A
}
```

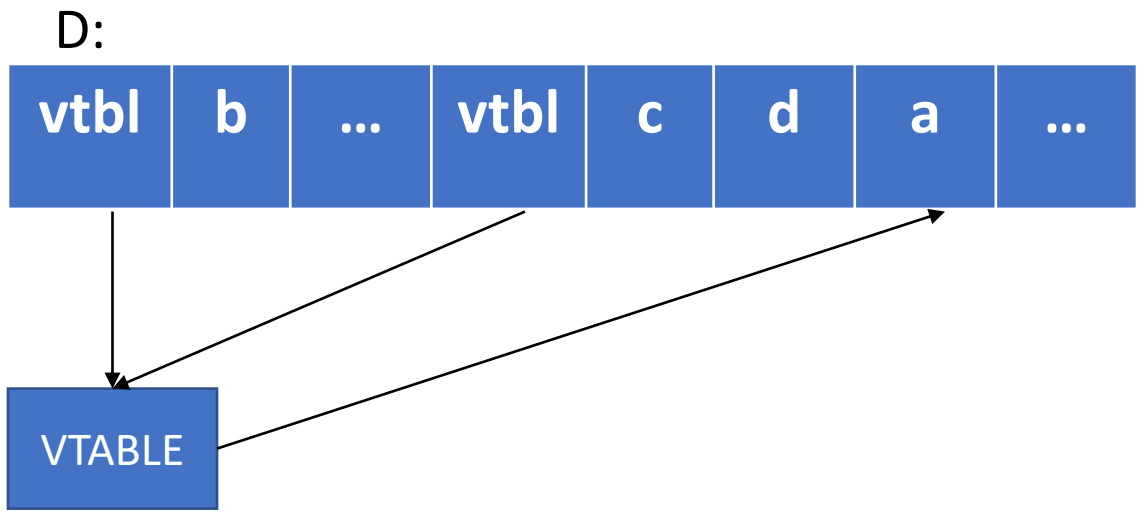


Виртуальное наследование

```
class A {
public:
    void f(){std::cout << "A\n";}
};
class B: public virtual A {
public:
    void f(){std::cout << "B\n";}
};
class C: public virtual A {
public:
    void f(){std::cout << "C\n";}
};
class D: public B,  public C {
public:
    int d = 5;
};
int main() {
    D d;
    d.A::f();    //A
    A& a = d;
    d.B::f();   //B
    static_cast<B>(d).A::f();   //A  , но d.f() неоднозначно
}
```

Как располагается в памяти?

```
class A {
public:
    int a = 3;
    void f(){std::cout << "A\n";}
};
class B: public virtual A {
public:
    int b = 4;
    void f(){std::cout << "B\n";}
};
class C: public virtual A {
public:
    int c = 5;
    void f(){std::cout << "C\n";}
};
class D: public B, public C {
public:
    int d = 5;
};
```

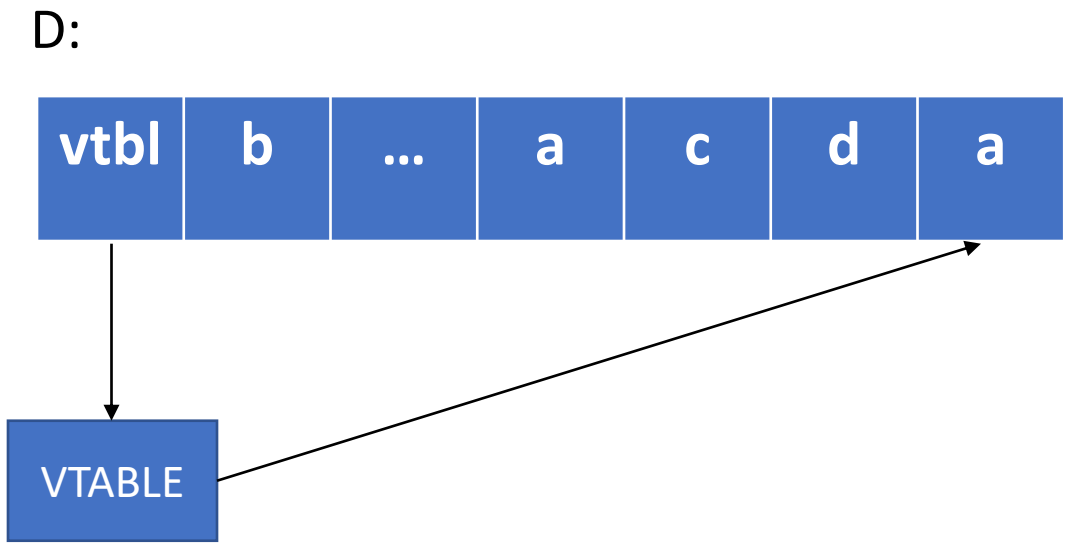


В Vtable перечислены адреса родительских классов

```
int main() {
    D d;
    std::cout << sizeof(d); //40
}
```

Если одно наследование virtual, а другое нет

```
class A {
public:
    int a = 3;
    void f(){std::cout << "A\n";}
};
class B: public virtual A {
public:
    int b = 4;
    void f(){std::cout << "B\n";}
};
class C: public A {
public:
    int c = 5;
    void f(){std::cout << "C\n";}
};
class D: public B, public C {
public:
    int d = 5;
};
```



Та же проблема ромбовидного наследования

Виртуальность и множественное наследование

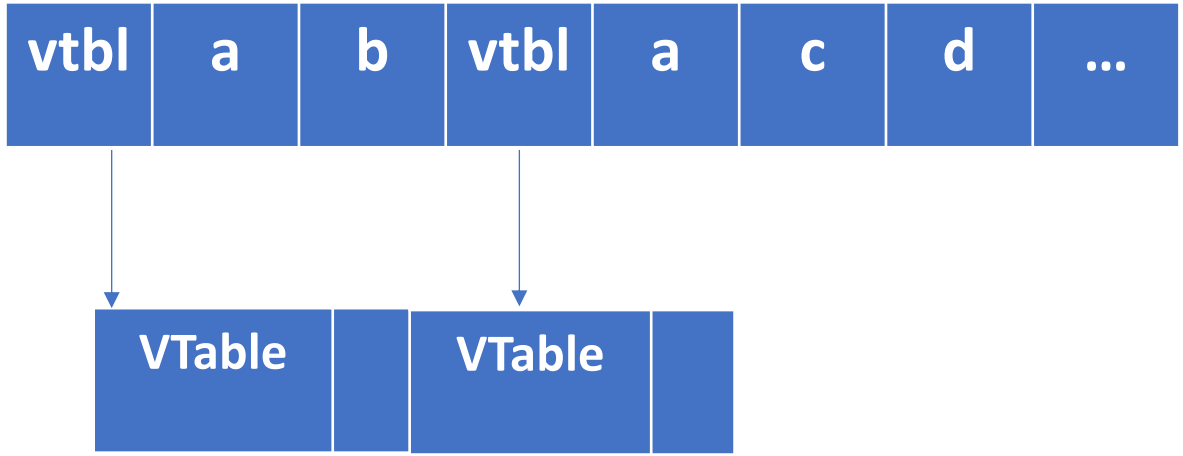
```
class A {
public:
    A() = default;
    virtual void print() const { std::cout << "print() from A" << std::endl; }
};
class B {
public:
    B() {}
    void print() const { std::cout << "print() from B" << std::endl; }
};
class C: public A, public B {
public:
    C(): A(), B() {}
    void print() const { std::cout << "print() from C" << std::endl; }
};
int main() {
    C c;
    A& a = c;
    a.print();   // ??
    B& b = c;
    b.print();   //??
}
```

Ромбовидное наследование с виртуальными функциями

```
class A {
public:
    int a = 2;
    virtual void f(){std::cout << "A\n";}
};
class B: public A {
public:
    int b = 2;
};
class C: public A {
public:
    int c = 4;
};
class D: public B,  public C {
public:
    int d = 5;
    void f() override {std::cout << "D\n";}
};

int main() {
    D d;
    d.f();    // ok
}
```

D d;

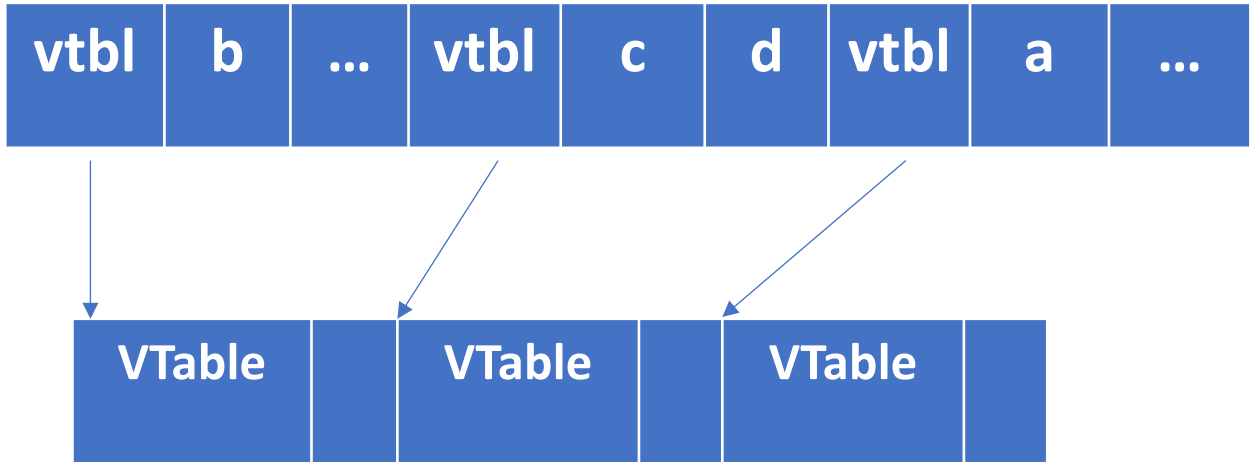
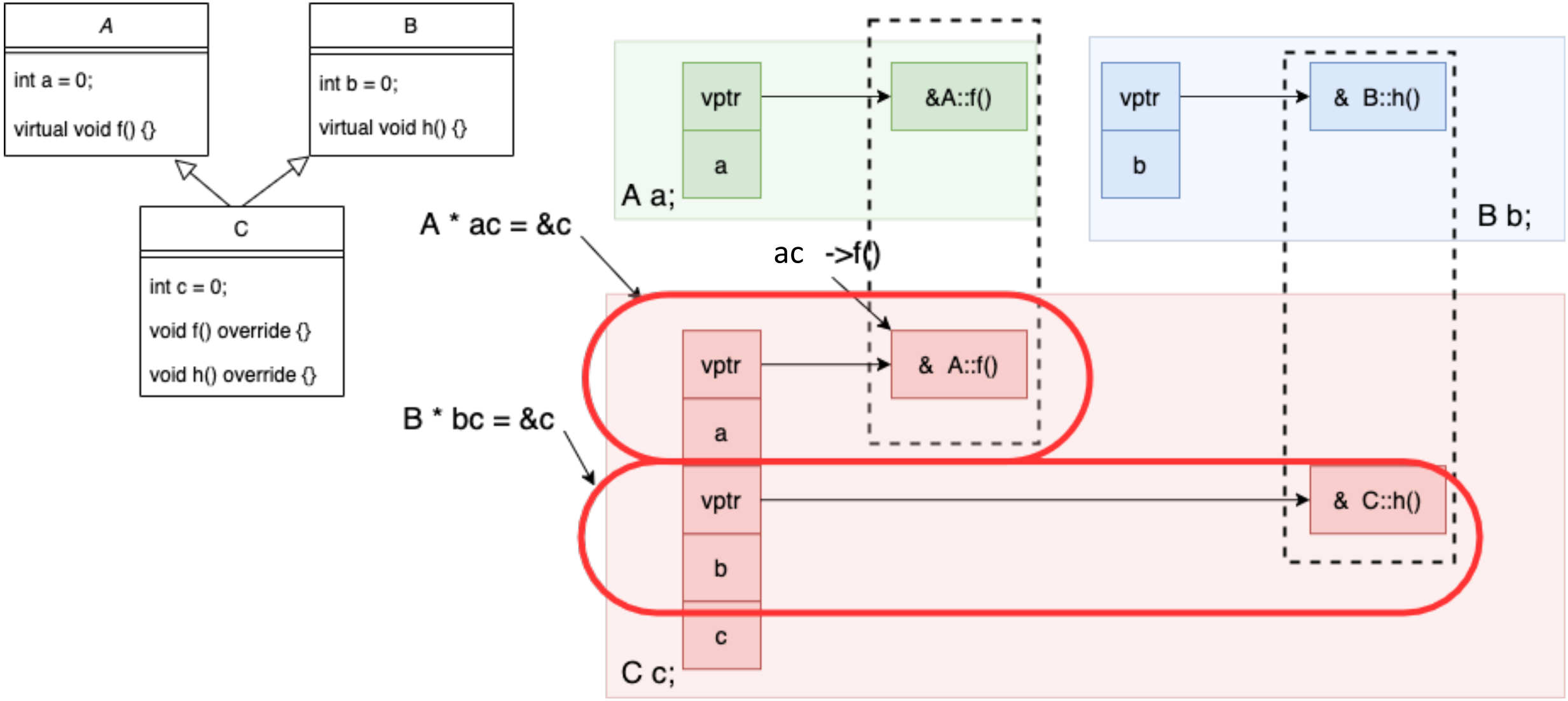


Виртуальное наследование с виртуальными функциями

```
class A {
public:
    int a = 3;
    virtual void f(){std::cout << "A\n";}
};
class B: public virtual A {
public:
    int b = 4;
};
class C: public virtual A {
public:
    int c = 5;
    void f(){std::cout << "C\n";}
};
class D: public B, public C {
public:
    int d = 5;
};

int main() {
    D d;
    d.f();
}
```

D d;



Vtable и множественное наследование